

```
Decimal('0.1')
>>> d + d + d
Decimal('0.3')
```

Integers don't have rounding errors, so it's always safe to pass them to `decimal.Decimal()`. Enter the following into the interactive shell:

```
>>> 10 + d
Decimal('10.1')
>>> d * 3
Decimal('0.3')
>>> 1 - d
Decimal('0.9')
>>> d + 0.1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'decimal.Decimal' and 'float'
```

But `Decimal` objects don't have unlimited precision; they simply have a predictable, well-established level of precision. For example, consider the following operations:

```
>>> import decimal
>>> d = decimal.Decimal(1) / 3
>>> d
Decimal('0.3333333333333333333333333333')
>>> d * 3
Decimal('0.9999999999999999999999999999')
>>> (d * 3) == 1 # d is not exactly 1/3
False
```

The expression `decimal.Decimal(1) / 3` evaluates to a value that isn't exactly one-third. But by default, it'll be precise to 28 significant digits. You can find out how many significant digits the `decimal` module uses by accessing the `decimal.getcontext().prec` attribute. (Technically, `prec` is an attribute of the `Context` object returned by `getcontext()`, but it's convenient to put it on one line.) You can change this attribute so that all `Decimal` objects created afterward use this new level of precision. The following interactive shell example lowers the precision from the original 28 significant digits to 2: